

Test Specification Document

Service Mobility Dashboard

1. Purpose

The purpose of this document is to define the tests required to verify that the Service Mobility Dashboard operates correctly, reliably, and securely, and to map the current automated test suite to the product's most important business flows.

The testing strategy focuses on validating the product's core business flows, business rules, and critical system behavior rather than achieving exhaustive line-by-line code coverage.

The testing approach is source-system agnostic at the application boundary: it validates behavior against normalized service_tasks data in Supabase rather than against a vendor-specific ERP API. The original business need arose in an SAP-based environment, but the same test strategy applies when another ERP, organizational database, or core operational system is mapped into the same application schema.

The primary areas under test are:

- Secure access to the operational dashboard.
- Loading service tasks for a selected workday.
- Displaying and grouping service tasks by vehicle on the map.
- Calculating and displaying optimized vehicle routes.
- Viewing detailed service-task information.
- Searching for a new service address through Google Places.
- Receiving and comparing vehicle-and-date recommendations.
- Preserving the read-only boundary of data mirrored from the upstream ERP or core operational system.
- Handling invalid input, missing data, and external-service failures.
- Correct Hebrew right-to-left presentation and operational usability.

2. Current Testing Strategy and Tools

Automated Unit Tests - Vitest

Vitest is used for deterministic server-side business logic. The committed unit suite currently verifies coordinate validation in getRecommendations(), including valid boundaries, ordinary valid coordinates, out-of-range values, NaN, and infinite values. Supabase is mocked so invalid client input can be proven to be rejected before any database access occurs.

Automated End-to-End Tests - Playwright

Playwright runs the real Next.js application and verifies the three highest-priority dispatcher flows: daily planning, new-service-task decision support, and secure logout. The tests use environment variables for test credentials rather than hard-coded accounts.

Manual Verification and Database Verification

Visual map behavior, edge cases, external-service failure states, and database permission rules are covered through Manual Verification or Database Verification when they are not represented by a committed automated test.

React Testing Library is installed in the project, but the current committed suite does not contain standalone component tests. Component behavior that is business-critical is currently exercised through Playwright or manual acceptance testing.

3. Current Automated Coverage

ID	Tool	Implemented Coverage	Status
UNIT-01	Vitest	Reject invalid recommendation coordinates before Supabase access; allow valid boundary and ordinary coordinates to continue.	Implemented
E2E-01	Playwright	Known-date deep link -> Login -> preserved date -> daily dashboard -> task markers/routes -> select task -> task tooltip; also checks for unexpected browser errors.	Implemented

ID	Tool	Implemented Coverage	Status
E2E-02	Playwright	Login -> Google Places address search -> draft marker -> seeded recommendation fixture -> distance ordering -> recommended day -> searched address preserved.	Implemented
E2E-03	Playwright	Login -> logout -> direct protected navigation -> redirected to login; operational dashboard remains inaccessible.	Implemented

Automated Test Environment Requirements

- Playwright loads .env.local explicitly because its configuration runs outside Next.js environment loading.
- E2E-01 and E2E-03 require E2E_DISPATCHER_EMAIL and E2E_DISPATCHER_PASSWORD.
- E2E-02 additionally requires E2E_SEARCH_ADDRESS and uses the live Google Places API. The committed test configuration uses the known search address "תל אביב, 10, דיזינגוף 10".
- E2E tests run against a real Next.js development server at localhost:3000.
- The test database must contain controlled service-task data for the dates and locations exercised by the tests. E2E-01 uses the known workday 2026-08-20 from the shared test dataset. E2E-02 uses a dedicated recommendation fixture refreshed by supabase/seed-e2e.sql before execution.

Before E2E-02 is run, supabase/seed-e2e.sql is executed manually in the Supabase SQL Editor by an authorized database operator. It removes the previous dedicated E2E recommendation row and creates a fresh candidate for the following day at the configured search location. This keeps the test deterministic while preserving the application's real four-day and 20 km business rules. The fixture is test setup data only and does not change the read-only permissions available to normal application users.

Controlled Test Data

The broader acceptance suite should include test data covering:

- Multiple vehicles and multiple tasks for the same vehicle.
- Tasks on several different dates.
- A vehicle with one task, exactly 12 routable tasks, and more than 12 routable tasks.
- Tasks with invalid coordinates and missing optional customer information.
- Tasks both inside and outside the 20 km recommendation radius.
- A workday containing no tasks.

4. Authentication and Authorization Tests

AUTH-01 - Unauthenticated User Attempts to Access Dashboard

Scenario: A user without a valid session navigates directly to the dashboard.

Expected Result:

- The user is redirected to /login.
- Operational service-task information is not rendered.

Coverage: Implemented indirectly in Playwright E2E-01, E2E-02, and E2E-03.

AUTH-02 - Preserve Requested Date During Authentication

Scenario: An unauthenticated user opens /?date=2026-08-20.

Expected Result:

- The user is redirected to the login page.
- The requested relative URL, including the date query string, is preserved.
- After successful login, the user returns to the requested workday.

Coverage: Implemented in Playwright E2E-01.

AUTH-03 - Successful Login

Scenario: An authorized dispatcher enters valid credentials.

Expected Result:

- Supabase authentication succeeds.
- The user reaches the protected dashboard.
- The daily operational view becomes accessible.

Coverage: Implemented in all three Playwright E2E flows.

AUTH-04 - Invalid Credentials

Scenario: A user enters an invalid email or password.

Expected Result:

- Login fails and the dashboard remains inaccessible.
- The generic message "Invalid email or password." is shown.
- The response does not reveal whether the email address exists.

Coverage: Manual acceptance test.

AUTH-05 - Empty Login Fields

Scenario: The login form is submitted with an empty email or password.

Expected Result:

- Required-field validation prevents successful login.
- No authenticated session is created.

Coverage: Manual/browser validation test.

AUTH-06 - Open Redirect Protection

Scenario: A login request contains an external or protocol-relative redirectTo value.

Expected Result:

- The application does not redirect the user to an external origin.
- The destination falls back to /.

Coverage: Manual verification; server-side protection is implemented.

AUTH-07 - Secure Logout

Scenario: An authenticated dispatcher selects the logout action and then attempts to revisit the dashboard.

Expected Result:

- The Supabase session is terminated.
- The user is redirected to /login.
- Direct navigation to the dashboard redirects back to /login.
- Operational dashboard content is not rendered after logout.

Coverage: Implemented in Playwright E2E-03.

5. Daily Task and Date Tests

DATE-01 - Default Date

Scenario: An authenticated dispatcher opens the dashboard without a date query parameter.

Expected Result:

- The dashboard uses today's date in the Asia/Jerusalem service timezone.
- Only tasks scheduled for that date are requested.

Coverage: Manual acceptance test; behavior is implemented in server code.

DATE-02 - Select Valid Date

Scenario: The dispatcher selects another date from the date controls.

Expected Result:

- The URL changes to /?date=YYYY-MM-DD.
- The server loads tasks for that selected date.
- Tasks from other dates are not displayed.

Coverage: Manual acceptance test.

DATE-03 - Invalid Date in URL

Scenario: The dashboard receives a malformed or impossible date such as /?date=2026-02-30.

Expected Result:

- The invalid value is rejected before the database query.
- The application falls back to today in the service timezone.
- The page remains usable.

Coverage: Manual verification; validation is implemented with isIsoDate().

DATE-04 - Previous and Next Day Navigation

Scenario: The dispatcher uses the previous-day or next-day control.

Expected Result:

- The date changes by exactly one calendar day.

- The corresponding workday is loaded.

Coverage: Manual acceptance test.

DATE-05 - Calendar Selection

Scenario: The dispatcher opens the calendar and selects a workday.

Expected Result:

- The selected date appears in the header.
- The dashboard navigates to and loads that date.

Coverage: Manual acceptance test.

DATE-06 - Date With No Tasks

Scenario: The dispatcher opens a valid date containing no service tasks.

Expected Result:

- The application does not crash.
- A Hebrew empty-state message is displayed.
- The dispatcher can continue navigating to other dates.

Coverage: Manual acceptance test.

6. Database and Task Retrieval Tests

DATA-01 - Retrieve Only Selected Day's Tasks

Scenario: The database contains tasks across multiple dates and getDailyTasks() is called for one date.

Expected Result:

- Only rows whose scheduled_date equals the requested date are returned.
- Rows are ordered by vehicle_id and then time_window for deterministic display.

Coverage: Manual verification; query behavior is implemented.

DATA-02 - Unauthenticated Server Action Request

Scenario: getDailyTasks() is invoked without valid authenticated claims.

Expected Result:

- No service-task records are returned.
- The result contains a "Not authenticated." error.

Coverage: Manual verification; authentication is re-checked inside the Server Action.

DATA-03 - Database Failure

Scenario: Supabase returns an error while daily tasks are being loaded.

Expected Result:

- The application returns an empty task list rather than crashing the route.
- A recoverable error is surfaced in the dashboard.
- The rest of the application remains navigable.

Coverage: Manual verification of the failure path.

7. Map and Vehicle Grouping Tests

MAP-01 - Group Tasks by Vehicle

Scenario: The selected day contains service tasks assigned to multiple vehicle_id values.

Expected Result:

- Tasks with the same vehicle_id are grouped together.
- Vehicle groups are sorted deterministically.
- Each vehicle receives a stable visual encoding.

Coverage: Manual verification.

MAP-02 - Valid Coordinates

Scenario: A task contains finite latitude and longitude within the valid ranges.

Expected Result:

- The task is considered routable and can be represented geographically.
- It may participate in route optimization.

Coverage: Covered indirectly by E2E-01 and the shared coordinate guard.

MAP-03 - Invalid Task Coordinates

Scenario: A database task has missing, non-finite, or out-of-range coordinates.

Expected Result:

- The task is excluded from route optimization and geographical bounds.
- The dashboard shows an informational warning for invalid-coordinate stops.
- Other valid tasks continue to function.

Coverage: Manual verification.

MAP-04 - Map Bounds

Scenario: Multiple valid service locations exist.

Expected Result:

- The map can fit its view around the valid service locations.
- Invalid-coordinate tasks do not corrupt the bounds calculation.

Coverage: Manual acceptance test.

8. Route Optimization Tests**ROUTE-01 - Vehicle With Multiple Stops**

Scenario: A vehicle has at least two valid routable tasks.

Expected Result:

- A Mapbox Optimization request is created for that vehicle.
- A usable route is drawn when Mapbox returns a valid trip.
- Stop numbers reflect the waypoint order returned by Mapbox.

Coverage: Partially automated through Playwright E2E-01, with response-handling behavior additionally verified manually.

ROUTE-02 - Vehicle With One Stop

Scenario: A vehicle contains only one valid task.

Expected Result:

- The task remains visible as a stop.
- No unnecessary Optimization API request is sent.
- No route line is required.

Coverage: Manual acceptance test.

ROUTE-03 - Exactly 12 Routable Stops

Scenario: A vehicle has exactly 12 valid coordinates.

Expected Result:

- All 12 stops are eligible for a single Mapbox Optimization request.
- No stop is treated as overflow.

Coverage: Manual verification.

ROUTE-04 - More Than 12 Routable Stops

Scenario: A vehicle has more than the Mapbox Optimization v1 limit of 12 valid coordinates.

Expected Result:

- At most the first 12 routable tasks are included in the optimization request.
- Additional valid tasks remain represented as marker-only overflow stops.
- The dashboard communicates the 12-point limitation to the user.

Coverage: Manual verification.

ROUTE-05 - Successful Mapbox Response

Scenario: Mapbox returns code Ok with a valid LineString geometry and waypoint indexes.

Expected Result:

- The route geometry is accepted and displayed.
- Distance and duration are retained for the route.
- Waypoint indexes are converted to human-readable stop numbers.

Coverage: Manual verification.

ROUTE-06 - Mapbox Failure or NoRoute

Scenario: The Optimization API fails, is unreachable, or returns a non-Ok code.

Expected Result:

- The failed vehicle falls back to stop markers without a route line.
- A Hebrew error toast identifies that route calculation failed for the vehicle.
- Other vehicles remain unaffected.
- The dashboard remains usable.

Coverage: Manual failure-path test.

ROUTE-07 - Malformed Optimization Response

Scenario: Mapbox responds without a usable trip geometry.

Expected Result:

- The response is treated as an optimization error.
- An invalid route is not drawn.
- The remaining dashboard functionality continues to operate.

Coverage: Manual verification.

9. Service Task Detail Tests

TASK-01 - Open Task Details

Scenario: The dispatcher selects a task from the sidebar or map.

Expected Result:

- The selected task is focused and a task tooltip is shown.
- Available operational information is displayed, including relevant fields such as installer, vehicle plate, customer name, phone, address, service window, note, and route stop number when available.

Coverage: Implemented at business-flow level in Playwright E2E-01.

TASK-02 - Missing Optional Fields

Scenario: A service task is missing optional information from the upstream enterprise source.

Expected Result:

- The UI does not expose literal null or undefined values.
- Unavailable detail rows are omitted or represented safely.
- The remaining task information stays readable.

Coverage: Manual acceptance test.

10. Address Search and Recent Search Tests

SEARCH-01 - Short Search Input

Scenario: The dispatcher enters fewer than three non-space characters.

Expected Result:

- No Google Places Autocomplete request is made.
- Autocomplete suggestions are cleared.

Coverage: Manual verification.

SEARCH-02 - Valid Israeli Address Search

Scenario: The dispatcher types a valid address with the Google Places API key configured.

Expected Result:

- Google Places Autocomplete is called after the debounce period.
- The request is localized to Hebrew and restricted to Israel.
- Valid place predictions are shown as suggestions.

Coverage: Implemented at business-flow level in Playwright E2E-02 using the live Google Places API.

SEARCH-03 - Select Suggested Address

Scenario: The dispatcher selects a Google Places prediction.

Expected Result:

- Place Details resolves latitude and longitude.
- The selected address remains in the search field.
- The suggestions close.
- The selected location appears on the map and becomes the draft location for recommendations.

Coverage: Implemented in Playwright E2E-02.

SEARCH-04 - Google Places Failure

Scenario: Autocomplete or Place Details fails or is aborted.

Expected Result:

- The application does not crash.
- Stale or invalid suggestions are not presented as a successful selection.
- The existing dashboard remains usable.

Coverage: Manual failure-path test.

SEARCH-05 - Recent Searches Display

Scenario: The address field is focused while empty and recent searches are stored in the browser.

Expected Result:

- A recent-search list is shown.
- Selecting a recent address reuses its stored coordinates without requiring a new Place Details lookup.

Coverage: Manual verification.

SEARCH-06 - Recent Search Limit and De-duplication

Scenario: More than five searches are selected, including a repeated address.

Expected Result:

- At most five recent searches are retained.
- A repeated address is moved to the newest position rather than duplicated.

Coverage: Manual verification.

SEARCH-07 - Unavailable Browser Storage

Scenario: localStorage is unavailable, throws, or cannot persist data.

Expected Result:

- Address search continues to work.
- Failure to persist recent searches does not crash the product.

Coverage: Manual verification of the failure path.

11. Recommendation Tests

REC-01 - Four-Day Recommendation Window

Scenario: A valid draft location is selected.

Expected Result:

- The recommendation query considers tomorrow through today + 4 days, inclusive.
- The window is based on the real current date in the service timezone, not the workday currently displayed on the dashboard.

Coverage: Manual verification; the business rule is implemented server-side.

REC-02 - 20 km Recommendation Radius

Scenario: Upcoming tasks exist both inside and outside 20 km of the searched location.

Expected Result:

- Only candidate tasks at a calculated distance of 20 km or less remain eligible.
- Tasks outside the radius are excluded.

Coverage: Manual verification.

REC-03 - Closest Task Per Vehicle and Date

Scenario: The same vehicle has several nearby tasks on the same scheduled_date.

Expected Result:

- Only the closest task for that vehicle/date combination is returned.
- Duplicate vehicle/date recommendation entries are not shown.

Coverage: Manual verification.

REC-04 - Recommendation Sorting

Scenario: Several valid recommendation candidates exist.

Expected Result:

- Recommendations are ordered from the shortest geographical distance to the longest.

Coverage: Implemented in Playwright E2E-02.

REC-05 - Invalid Database Coordinates

Scenario: Upcoming database tasks contain invalid geographical coordinates.

Expected Result:

- Invalid records are excluded from distance calculations.
- Valid candidates continue to be processed.

Coverage: Manual verification.

REC-06 - No Recommendations

Scenario: No upcoming service task falls within the 20 km radius.

Expected Result:

- No false candidate is generated.
- The dashboard remains usable and the searched location can still be inspected.

Coverage: Manual acceptance test.

REC-07 - Select Recommendation and Compare Workday

Scenario: The dispatcher chooses a recommendation.

Expected Result:

- The dashboard navigates to the recommendation's scheduled_date.
- The searched address remains available on the map for comparison with the existing schedule.

Coverage: Implemented in Playwright E2E-02.

REC-08 - Recommendation Database Failure

Scenario: Supabase fails while recommendation candidates are loaded.

Expected Result:

- No incorrect recommendation data is shown.
- The rest of the dashboard remains operational.

Coverage: Manual verification of the failure path.

REC-09 - Unauthenticated Recommendation Request

Scenario: getRecommendations() is invoked without authenticated claims.

Expected Result:

- No recommendation data is returned.
- The Server Action does not expose operational task data.

Coverage: Manual verification; the server-side authentication check is implemented.

12. Input Validation Tests**VAL-01 - Invalid Search-Date Input**

Scenario: The date query parameter is malformed or does not represent a real calendar date.

Expected Result:

- The value is rejected before being used as a PostgreSQL date filter.
- The dashboard falls back to a valid service date.

Coverage: Covered by DATE-03 specification.

VAL-02 - Invalid Recommendation Coordinates

Scenario: getRecommendations() receives coordinates outside valid latitude/longitude ranges, NaN, or infinite values.

Expected Result:

- The function returns an empty recommendation list.
- Supabase is not initialized or queried for invalid client input.

Coverage: Implemented in Vitest UNIT-01.

VAL-03 - Coordinate Boundary Values

Scenario: getRecommendations() receives latitude ± 90 , longitude ± 180 , zero coordinates, or ordinary valid coordinates.

Expected Result:

- The values pass coordinate validation.
- Execution proceeds to the protected server flow.

Coverage: Implemented in Vitest UNIT-01.

VAL-04 - External Redirect Input

Scenario: redirectTo is external, malformed as a protocol-relative URL, or otherwise not a same-origin relative path.

Expected Result:

- The supplied destination is rejected.
- The post-login destination becomes /.

Coverage: Covered by AUTH-06 specification; server validation is implemented.

13. Database Permission and RLS Tests

The service_tasks table is a read-only mirror of normalized operational data from an upstream enterprise source. Database-level verification must confirm both privilege revocation and Row Level Security behavior; UI restrictions alone are not sufficient.

DB-01 - Authenticated Read Access

Scenario: An authenticated application role selects service_tasks.

Expected Result:

- SELECT is permitted through the explicit authenticated read policy.
- Rows required by the dashboard are readable.

Coverage: Database verification test.

DB-02 - Anonymous Read Is Denied

Scenario: The anon role attempts to select service_tasks.

Expected Result:

- Anonymous SELECT access is denied.
- Operational service-task data is not exposed to unauthenticated database clients.

Coverage: Database verification test; enforced by migration 0005.

DB-03 - Prevent INSERT

Scenario: An anon or authenticated application role attempts to insert a service task.

Expected Result:

- PostgreSQL/Supabase rejects the write.

Coverage: Database verification test; privileges are revoked by migration 0003.

DB-04 - Prevent UPDATE

Scenario: An anon or authenticated application role attempts to update a service task.

Expected Result:

- PostgreSQL/Supabase rejects the write.

Coverage: Database verification test; UPDATE policies are removed and privileges revoked.

DB-05 - Prevent DELETE

Scenario: An anon or authenticated application role attempts to delete a service task.

Expected Result:

- PostgreSQL/Supabase rejects the write.

Coverage: Database verification test; privileges are revoked by migration 0003.

14. Hebrew UI and Operational UX Tests

UI-01 - Hebrew Right-to-Left Interface

Scenario: The authenticated dashboard is opened in the current product version.

Expected Result:

- Operational application text is presented in Hebrew.
- The document and primary interface use right-to-left direction.
- Map interaction remains usable in the RTL layout.

Coverage: Manual acceptance test.

UI-02 - LTR Operational Values Inside RTL UI

Scenario: The Hebrew interface displays phone numbers, vehicle identifiers, addresses containing Latin characters, or time values.

Expected Result:

- Values remain visually readable and are not unintentionally reversed.

Coverage: Manual visual test.

UI-03 - Loading States

Scenario: Route calculation or recommendation loading is in progress.

Expected Result:

- The interface displays appropriate Hebrew progress feedback such as route or recommendation loading text.
- The application remains responsive while asynchronous work is pending.

Coverage: Partially exercised by E2E-01 and E2E-02, which wait for these states to complete.

UI-04 - Recoverable Error Feedback

Scenario: Daily data loading or a vehicle route fails.

Expected Result:

- The user receives visible feedback through inline messaging and/or a toast.
- Unrelated dashboard functionality is not unnecessarily blocked.

Coverage: Manual failure-path test.

UI-05 - Vehicle Focus and Visibility Controls

Scenario: The dispatcher focuses a vehicle or hides it from the current map view.

Expected Result:

- The requested visual filtering occurs without re-querying Supabase.
- Other vehicle data remains available and can be restored.

Coverage: Manual acceptance test.

15. Important Edge Cases

- A valid workday with zero service tasks.
- A vehicle with one routable task.
- A vehicle with exactly 12 routable tasks.
- A vehicle with more than 12 routable tasks.
- Several vehicles operating in the same geographical area.
- One invalid-coordinate task while all other tasks remain valid.
- Missing optional customer information.
- Mapbox route failure for one vehicle while other vehicle routes succeed.
- Rapid date navigation.
- Rapid changes to the address-search query while previous Google Places requests are pending.
- Google Places Place Details failure after a prediction is selected.
- Unavailable localStorage for recent searches.
- An expired authentication session.

16. Core End-to-End Business Flows**E2E-01 - Daily Planning Flow - Implemented**

1. Dispatcher opens the known workday /?date=2026-08-20, which contains multiple valid service tasks in the shared test dataset.
2. Unauthenticated access redirects to Login while preserving the requested date for the post-login return.
3. Dispatcher signs in with test credentials.
4. After authentication, the dashboard returns to 2026-08-20 and the Hebrew daily-planning heading is visible.
5. Service stop markers are present.
6. The test waits for route optimization to finish.
7. Dispatcher selects a task from the sidebar.
8. Task tooltip is displayed.
9. No unexpected browser-side JavaScript errors occur.

Success Criteria: The dispatcher can reach and use the daily operational view without unexpected browser errors.

E2E-02 - New Service Task Decision-Support Flow - Implemented

1. Dispatcher signs in.
2. Dispatcher searches a configured address using Google Places autocomplete.
3. Dispatcher selects a returned place prediction.
4. The resolved draft location appears on the map.
5. The system loads upcoming vehicle/date recommendations.
6. At least one seeded recommendation is displayed.
7. Recommendation distances are verified to be sorted from nearest to farthest.
8. Dispatcher selects the first recommendation.

9. Dashboard navigates to the recommendation date.
10. The searched address remains visible for comparison.
11. No unexpected browser-side JavaScript errors occur.

Success Criteria: The dispatcher can evaluate a new-task assignment using the live application flow without modifying source-system-mirrored operational data.

E2E-03 - Secure Logout Flow - Implemented

1. Dispatcher signs in.
2. Protected dashboard is accessible while authenticated.
3. Dispatcher selects Logout.
4. The session ends and the browser returns to Login.
5. Direct navigation to the protected dashboard redirects back to Login.
6. Operational dashboard content is not rendered.
7. No unexpected browser-side JavaScript errors occur.

Success Criteria: Operational information is inaccessible after logout.

17. Test Execution and Completion Criteria

Commands

Automated tests are exposed through the project scripts:

- `npm test` - runs the Vitest unit suite.
- `npm run test:watch` - runs Vitest in watch mode during development.
- `npm run test:e2e` - runs the Playwright end-to-end suite against a real Next.js server.

E2E setup - before running the suite, execute `supabase/seed-e2e.sql` in the Supabase SQL Editor to refresh the dedicated E2E-02 recommendation fixture.

Completion Criteria

Final validation snapshot (1 September 2026): `npm run lint` passed; Vitest passed 12/12 unit tests; Playwright passed 3/3 E2E flows; and `npm run build` completed successfully.

The product is considered ready for final submission when:

- The committed Vitest and Playwright suites pass in the project's intended development/test environment.
- The three high-priority E2E business flows pass.
- Authentication protections and secure logout behavior are verified.
- Read-only database protections and RLS behavior are verified.
- Recommendation input validation and core recommendation business rules are verified.
- Important route and external-service failure states are manually checked.
- The Hebrew RTL interface passes a final visual and usability smoke test.
- No critical or high-severity defects remain open.
- The production deployment passes a final smoke test before submission.